

Relatório - Sistema de Matchmaking

Samuel Boldrini, Luis Teodoro Fonseca

Maio de 2026

1 Introdução

Este projeto implementa um sistema de matchmaking para jogos online utilizando a linguagem C++. O sistema foi desenvolvido utilizando programação orientada a objetos, modularização com arquivos `‘.hpp’` e `‘.cpp’`, arrays estáticos e algoritmos de ordenação implementados manualmente. O objetivo principal do sistema é armazenar jogadores em uma fila de espera, ordenar os jogadores por score e formar grupos de jogadores com níveis de habilidade semelhantes

2 Ordenação

Foram implementados dois algoritmos de ordenação: Insertion Sort: algoritmo simples baseado em inserção. Para cada posição do array, o elemento é comparado com os anteriores até encontrar sua posição correta. O algoritmo possui custo $O(n^2)$ no pior caso. Merge Sort: algoritmo baseado em divisão e conquista. O array é dividido recursivamente em partes menores, que posteriormente são intercaladas de forma ordenada utilizando a função merge. O custo do merge sort é $O(n \log n)$.

A recursão foi utilizada na implementação do merge sort. O método `‘mergeSort(left, right)’` divide o array recursivamente em duas metades até que cada subarray possua apenas um elemento. Em seguida, as partes são combinadas ordenadamente pela função `‘merge’`.

A ordenação dos jogadores foi utilizada para facilitar a formação de grupos válidos. Como os jogadores ficam organizados por score crescente, basta analisar grupos consecutivos de tamanho `‘groupSize’`. Dessa forma, para verificar se um grupo é válido, basta calcular: $\text{maior_score} - \text{menor_score} \leq \delta$. Como o array está ordenado, o menor score é o primeiro elemento do grupo e o maior score é o último. Isso reduz significativamente a complexidade da busca.

3 Custo Computacional

Segue abaixo as principais operações do sistema, junto de sua respectiva complexidade computacional.

Operação	Complexidade
Inserção	$O(1)$
Remoção	$O(n)$
Insertion Sort	$O(n^2)$
Merge Sort	$O(n \log(n))$
Formação de Grupo	$O(n)$
Recuperação de Jogadores	$O(n)$

4 Merge x Insertion

Os algoritmos MergeSort e InsertionSort levam tempos diferentes para ordenar a mesma lista. Segue abaixo uma tabela com o tempo em milissegundos levado por cada um dos algoritmos para ordenar a lista de acordo com o tamanho

	10.000	50.000	100.000
Merge	32	177	360
Insertion	1.745	43.064	170.403

Como podemos perceber, o MergeSort é muito mais eficiente que o InsertionSort, com o InsertionSort demorando 470 vezes mais para o tamanho máximo da lista.

Essa diferença é facilmente explicável, visto que o MergeSort tem complexidade $O(n \log(n))$ enquanto o InsertionSort tem complexidade $O(n^2)$.

Os testes foram realizados criando amostras aleatórias de jogadores usando a biblioteca <random>, depois calculando o tempo do merge sort (usando <chrono>), depois embaralhando a lista novamente com o método shuffle (novamente usando <random>) e calculando o tempo do InsertionSort.

O código dos testes realizados pode ser encontrado no arquivo 'time.cpp'.